

AI Application Architecture

Case study: Nova — an AI chat application

Nova is an AI-powered chat application (Next.js frontend) that lets a user send a message and receive a response from one of four LLM providers — Claude, ChatGPT, DeepSeek, or Gemini — selected through a backend API Router. Nova also uses Supabase for authentication and database, and Stripe for subscription payments. The diagram below shows the full path a message follows, plus the platform services the Backend talks to.

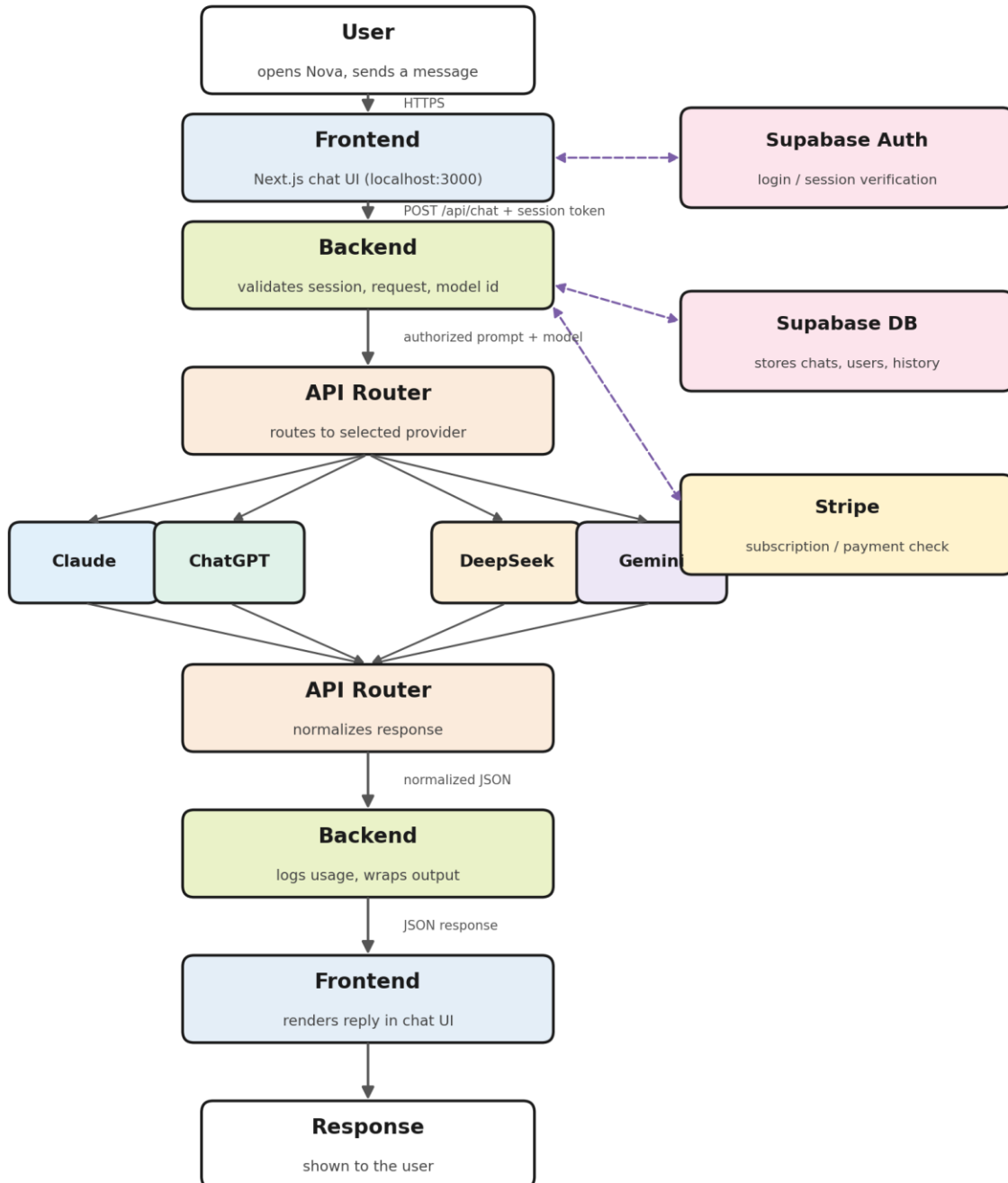


Figure 1. Core flow: User → Frontend → Backend → API Router → Claude/ChatGPT/DeepSeek/Gemini → back → Response. Side services: Supabase Auth, Supabase DB, and Stripe, all connected to the Backend.

Request Flow

The user logs in (Supabase Auth issues a session token) and sends a message with a chosen model. The Frontend POSTs the message, model id, and session token to the Backend. The Backend verifies the session against Supabase Auth, checks the user's subscription status via Stripe (for paid models/usage limits), and — if authorized — saves the message to Supabase DB before formatting the prompt and passing it, with the matching API key, to the API Router, which forwards it to the selected provider.

Response Flow

The selected provider returns a completion to the API Router, which normalizes it into one consistent shape regardless of which of the four models replied. The Backend receives this, logs the response and token usage to Supabase DB (for chat history and billing), and sends the final JSON back to the Frontend, which renders it in the chat UI.

Error Handling

- Auth errors: an expired or invalid Supabase session is rejected by the Backend before any LLM or Stripe call is made, and the Frontend redirects the user to log in again.
- Payment/subscription errors: if Stripe reports a lapsed subscription or a usage-limit breach, the Backend blocks the request with a clear “upgrade required” response instead of calling the LLM API.
- Provider failures: if a provider (e.g. DeepSeek) times out or errors, the API Router catches it and returns a clean error — it doesn't crash the request or leak provider-specific details.
- Database write failures: if a Supabase DB write fails (e.g. saving chat history), the Backend still returns the model's response to the user and logs the failure separately, so a DB hiccup never blocks the chat itself.
- Frontend fallback: if the Backend is unreachable, the Frontend shows a connection-error state and lets the user retry.

API Integration

All secrets — the four LLM API keys, the Supabase service key, and the Stripe secret key — live only on the Backend; the Frontend only ever holds a public Supabase anon key and a session token, so no sensitive credential is ever exposed to the browser. Supabase handles auth (session issuing/verification) and Postgres storage (users, chat history, subscription status), Stripe handles payment/subscription state via webhooks that update that same Supabase subscription record, and the API Router is the single integration point for all four LLM providers — keeping each concern (identity, billing, model access) in its own well-defined layer.